Reviewer Pack — Minimal Rank of Configuration Spaces vs Communication Comple…

Entry ac3b5aa63bfb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 07:51:03 UTC

Minimal Rank of Configuration Spaces vs Communication Complexity for Disjunctive Normal Form

Entry ID: `ac3b5aa63bfb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 07:51:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Configuration Space Theory) **Field B** (complexity object): Communication Complexity: Disjunctive Normal Form

Statement:

> ['For any given n-vertex graph G, the minimal rank of its configuration space is upper-bounded by a function of the communication complexity for the disjunction of all possible subsets of vertices in G.', 'Formally, for every n ≤ 40, there exists a constant C(n) such that for any graph G with n vertices, rank(config_space(G)) ≤ C(n) * comm_complexity(G).', 'The communication complexity for the disjunction of subsets is defined as the minimum number of bits required to communicate information between two players in order to determine whether the intersection of their input sets is non-empty.']

Rationale (proposer's reasoning):

> ['Configuration spaces are known to capture geometric and topological properties of a system, while communication complexity measures the amount of information that needs to be exchanged between parties. This conjecture aims to find a connection between these two concepts by exploring the relationship between the minimal rank of configuration spaces and the communication complexity for disjunctive normal forms.', 'If proven true, this would provide new insights into the interplay between geometric/topological properties and computational complexities.']

Taxonomy category: `CONFIG_SPACE_COMM_COMPLEXITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `4d0b6ca0e6487558`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across all 30 seeds and n ≤ 40 graphs, the mean ratio of the minimal rank of the configuration space to the communication complexity for the

disjunction of subsets is less than or equal to a constant C(n), with no seed producing a ratio greater than 1.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "configuration space theory" AND "communication complexity" AND "disjunctive normal form" - "minimal rank" OF graph configuration spaces AND relationship with communication complexity for disjunctive normal form" - "algebraic topology" in configuration spaces AND bounds on communication complexity via disjunctive normal form

Top relevant hits considered: - [http://arxiv.org/abs/2604.09703v1] Cayley Graph Optimization for Scalable Multi-Agent Communication Topologies - [http://arxiv.org/abs/1312.7368v2] Totally normal cellular stratified spaces and applications to the configuration space of graphs - [http://arxiv.org/abs/2503.22997v1] Disjunctive Complexity - [http://arxiv.org/abs/math/0612591v2] Associahedron, cyclohedron, and permutohedron as compactifications of configuration spaces - [http://arxiv.org/abs/math/9804001v2] New invariant tensors in CR structures and a normal form for real hypersurfaces at a generic Levi degeneracy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_random_graph(n):
    graph = []
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                graph.append((i, j))
    return graph

def config_space(graph):
    n = len(graph)
    subgraphs = [set() for _ in range(2**n)]
    for subset in range(2**n):
        subgraph = {edge for edge in graph if all(edge[i] in (subset >> i) & 1 for i in range(n))}
        subgraphs[subset].update(subgraph)
```

```
    return max(len(subgraph) for subgraph in subgraphs)

def communication_complexity(graph):
    n = len(graph)
    subsets = [set() for _ in range(2**n)]
    for subset in range(2**n):
        for edge in graph:
            if any(edge[i] in (subset >> i) & 1 for i in range(n)):
                subsets[subset].add(edge)
    max_size = max(len(subset) for subset in subsets)
    return math.ceil(math.log(max_size, 2))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    graph = generate_random_graph(n)
    rank = config_space(graph)
    comm_complexity = communication_complexity(graph)
    ratio = Fraction(rank, comm_complexity) if comm_complexity != 0 else float('inf')
    return {
        "metric_name": "Rank/CommComplexityRatio",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": ratio <= 1.5,
        "counterexample": "" if ratio <= 1.5 else f"n={n}, rank={rank}, comm_complexity={comm_comp
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30*31, 2))
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all("conjecture_holds": True for result in results):
        mean_ratio = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = next(result["counterexample"] for result in results if not result["conjec
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f869796c.py", line 91, in <module>
    result = run_trial(seed)
```

```
            ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f869796c.py", line 56, in run_trial
    rank = config_space(graph)
           ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f869796c.py", line 33, in config_space
    subgraph = {edge for edge in graph if all(edge[i] in (subset >> i) & 1 for i in range(n))}
                                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f869796c.py", line 33, in <genexpr>
    subgraph = {edge for edge in graph if all(edge[i] in (subset >> i) & 1 for i in range(n))}
                                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: argument of type 'int' is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating the conjecture's support conditions. | next: Investigate and fix the error in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15178
2	preregistration	ollama_remote	glm4:latest	0	0	9543
3	novelty	ollama_remote	glm4:latest	0	0	8663
4	novelty	ollama_remote	glm4:latest	0	0	8973
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29796
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10292
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9550
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7828
9	judge	ollama_remote	glm4:latest	0	0	11614

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 111438 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ac3b5aa63bfb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/ac3b5aa63bfb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ac3b5aa63bfb.tar.gz` (if generated)